

Certificateless Signature Scheme With Batch Verification for Secure and Privacy-Preserving V2V Communications in VANETs

Qian Wu [✉], Lei Zhang [✉], *Member, IEEE*, Yafang Yang [✉], and Kim-Kwang Raymond Choo [✉], *Senior Member, IEEE*

Abstract—The importance of designing efficient and secure vehicular ad hoc networks (VANETs), for example to address pressing traffic-related challenges in busy cities, is crucial but challenging. For example, existing approaches may suffer from vehicle-to-vehicle (V2V) communication-related limitations such as certificate management, key escrow, and performance. Seeking to contribute to the knowledge gap, we propose a new secure and privacy-preserving scheme for V2V communications. Our proposal is based on a new provably secure certificateless signature scheme with batch verification, which eliminates the need for certificates while ensuring better efficiency. Our performance and security evaluations demonstrate that the proposed approach overcomes our previously discussed limitations while achieving both message authentication and conditional privacy. In addition, we also shown that it incurs low overhead.

Index Terms—Batch verification, privacy, security, vehicle ad hoc networks.

I. INTRODUCTION

THE increase in vehicle volumes has partly contributed to traffic-related challenges (e.g., traffic congestion), and resulted in efforts to design intelligent transport systems (ITS). A key component of ITS is vehicle ad-hoc network (VANET) [1], which generally comprises three main entities (i.e., trusted authority (TA), roadside units (RSUs), and participating vehicles). VANETs employ two primary communication modes, namely: vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communications. The former (V2V) involves sharing traffic-related messages (TRMs) such as location, speed, and vehicle directional information, and the latter (V2I) allows vehicles to interact with RSUs. For instance, the Department of Transportation and Roads can collect TRMs from RSUs and

use such messages to facilitate decision-making (e.g., traffic redirection or road closure).

In any technological deployments, including VANETs, one needs to consider the underpinning security and privacy challenges [1], [2], [3] and implications. For instance, VANETs rely on wireless networks to facilitate V2V and V2I communications. Consequently, the deployment may be vulnerable to a broad range of common attacks that can result in the interception, modification, and replay of transmitted TRMs. In other words, it is crucial to ensure the integrity, authenticity, and non-repudiation of TRMs in VANETs. In addition, TRMs sent over the insecure medium may contain sensitive information, including the true identity and real-time position of a vehicle. Such information could be exploited by attackers, including nation states or organized crime groups, in various nefarious activities. For example, organized crime groups may hijack a vehicle and use the vehicle to (intentionally) broadcast fake messages to throw off law enforcement surveillance; hence, their identities must be disclosed. These risks reinforce the importance of designing secure and robust solutions that can help preserve users' privacy in VANETs, although in some context the privacy of a vehicle should be conditional.

The pseudonym technique, i.e., the technique that is based on digital signature and pseudonym, is a widely used technique to address the above security and privacy issues in VANETs [6]. Typically, in a secure vehicular communication scheme based on the pseudonym technique, vehicular messages are signed by the vehicles to ensure the integrity, authenticity and non-repudiation of messages. Another vehicle will accept the received message if the corresponding signature can successfully pass the checks undertaken using the associated signature verification algorithm. As to the privacy of a vehicle, it is achieved by frequently changing its pseudonyms, for example using anonymous certificates in traditional PKI-based cryptosystem (TPBC) [7] and pseudo-identity public keys in identity-based public key cryptosystem (IDPKC) and certificateless public key cryptosystem (CLPKC) [6], [8]. Existing digital signature schemes are mainly designed for TPBC, IDPKC and CLPKC, among which CLPKC is used to mitigate the underpinning certificate management limitation in TPBC without the associated key escrow challenge in IDPKC. This paper focuses on secure and privacy-preserving schemes (S&PPSSs) for V2V communications designed in CLPKC.

Manuscript received 9 June 2023; revised 23 July 2024; accepted 13 August 2024. Date of publication 16 August 2024; date of current version 14 March 2025. This work was supported in part by the NSF of China under Grant 62372177. The work of Kim-Kwang Raymond Choo was supported only by the Cloud Technology Endowed Professorship. (*Corresponding author: Lei Zhang.*)

Qian Wu and Lei Zhang are with the Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200062, China, and also with the Engineering Research Center of Software/Hardware Co-design Technology and Application, Ministry of Education, East China Normal University, Shanghai 200062, China (e-mail: leizhang@sei.ecnu.edu.cn).

Yafang Yang is with the School of Computer Science, Fudan University, Shanghai 200433, China.

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78249 USA.

Digital Object Identifier 10.1109/TDSC.2024.3445164

In addition the previously discussed security and privacy challenges, the issue of efficiency in VANETs is also an important design and deployment consideration. For example, once a vehicle receives TRMs from surrounding vehicles, it needs to process and respond promptly. According to [9], each vehicle has to broadcast TRMs every 100-300 milliseconds in VANETs. Therefore, during peak traffic, each vehicle should verify a large amount of message-signature pairs in a short period of time. If these pairs are verified one by one, it would be inefficient and potentially leads to a large message loss rate. Hence, the utility of techniques such as batch verification [10] and aggregate signature [11] has been explored. The former allows the verifier to verify multiple signatures simultaneously, while the latter combines multiple signatures into a single signature for subsequent verification. While both batch verification and aggregate signature can significantly reduce the cost of verification, aggregate signature does not guarantee that each signature (prior to the aggregation) is valid, unlike in the case of batch verification [12]. Therefore, compared with aggregate signature, batch verification can provide a higher security level.

A. Related Approaches

A number of S&PPSs for V2V communication have been proposed which can be broadly divided into three categories. The first category employs pseudonym technique, the second relies on group signatures, and the third involves ideal tamper-proof devices (TPDs) [13], [14]. Although the schemes in the second category can efficiently decrease the need for preloading a large number of pseudonyms in frequent pseudonym changing, the limitation is membership revocation that leads to much higher verification costs, storage costs, and communication overheads (in comparison to schemes in the first category) [15]. Schemes in the third category generally assume that a global secret is saved to each vehicle's TPD, and an attacker cannot obtain any information about the secret. However, as discussed in [16], this assumption is too strong. Thus, here we focus on schemes in the first category.

Hubaux et al. [17] proposed a concrete S&PPS for V2V communications in TPBC. To mitigate the underpinning certificate management limitation, many other S&PPSs for V2V communications have been designed in IDPKC (e.g., [18], [19], [20]), in which the public key of an entity (e.g., vehicle) is just its identity. However, schemes designed in IDPKC generally suffer from the key escrow problem. That is, an entity (called private key generator) used to issue private keys for the entities in the system knows the private key of any entity in the system.

CLPKC is designed to tackle the issue of key escrow in IDPKC. In CLPKC, a trusted third-party known as the key generation center (KGC) assists an entity in generating its private key. However, in CLPKC, the KGC only generates a partial private key for the entity. The entity's complete private key is derived by combining the partial private key with a secret value chosen by the entity itself. This ensures that the KGC does not possess the ability to access the entity's full private key, thereby eliminating the key escrow problem.

In recent years, several S&PPSs for V2V communications have been designed in CLPKC, as well as those based on certificateless aggregate signatures or certificateless signature schemes (CLSSs) with batch verification. Similarly, a large number of certificateless aggregate signature schemes have been proposed in the literature (e.g., [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34]), with some already utilized for S&PPSs in VANET V2V communications (e.g., [25], [26], [27], [28], [29], [30], [31], [32], [33], [34]). To the best of our knowledge, the most efficient and provably secure scheme is the one proposed in [24]. It is worth noting that several of these efficient certificateless aggregate signature schemes were later found to be insecure (e.g., [21], [22], [23], [25], [26], [27], [28], [29], [30], [33]). We also observe that the security of several of these schemes is proven based on the forking lemma (e.g., [23], [25], [26], [27], [28], [29], [30], [33]). It had been shown that the forking lemma is not applicable to certificateless aggregate signature schemes. In other words, a certificateless aggregate signature scheme that is proven secure using the forking lemma may actually be insecure (e.g., [31], [32], [34]). For CLSSs with batch verification, the provably secure and most efficient one is the scheme in [35]. We note that the most recent CLSSs with batch verification are given in [36], [37]. However, the scheme in [36] was later found to be insecure. The scheme in [37] is based on a CLSS. The authors of [37] proved that the CLSS is secure and showed that the signatures generated using this CLSS can be verified in a batch using a specific batch verification technique. However, the authors of [37] do not discuss whether the batch verification technique they used is secure.

B. Our Contributions

Motivated by the prior discussions, we propose an efficient secure and privacy-preserving scheme for V2V communications (ES&PPS) based on a new provably secure CLSS with batch verification.

For our CLSS with batch verification (see Section III), since it is designed in CLPKC, it inherits the advantages of certificate management problem freeness and key escrow freeness of CLPKC. Furthermore, it is able to verify multiple message-signature pairs simultaneously. Compared with the latest provably secure CLSS with batch verification, our new proposal has much lower computational and communication costs. The proposed CLSS and the pseudonym technique are then used as a building block for our proposed ES&PPS (see Section IV). We then evaluate the security and performance of our proposed scheme (see Sections V and VI) to demonstrate that it achieves message authentication, conditional privacy, certificate management problem freeness, fast response, key escrow freeness properties defined in Section II-B. Specifically, message authentication, certificate management freeness, fast response, key escrow freeness properties of our scheme are guaranteed by our CLSS with batch verification, and conditional privacy is realized using a secure symmetric key encryption scheme and the pseudonym technique.

In the next Section II, we will briefly introduce the background materials.

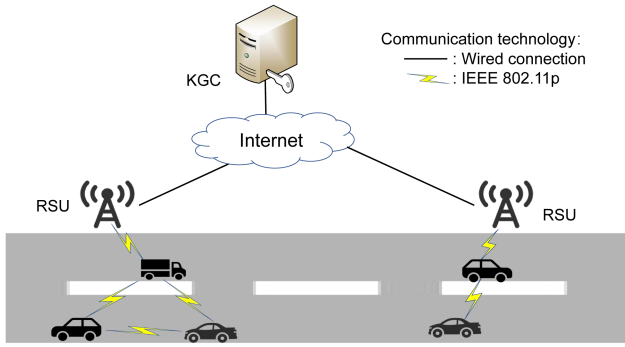


Fig. 1. System architecture.

II. BACKGROUND

A. System Model

As shown in Fig. 1, our system consists three types of entities, namely, KGC, RSU and Vehicle.

- **KGC:** It is a TA that generates the system parameters and partial private keys for the vehicles. The KGC can also track down any malicious vehicle that attempt to spread fake messages.
- **Vehicle:** Each vehicle is equipped with an on-board unit (OBU) that enables wireless communication with nearby vehicles and/or RSUs.
- **RSU:** RSUs are located at the side of the road. Each vehicle can establish a connection with the KGC via an RSU.

B. Design Goals

To address the security and privacy challenges discussed in Section I, our ES&PPS as well as any S&PPS for V2V communications in VANETs must satisfy the following properties.

- 1) **Message Authentication:** Each TRM should be from a legitimate vehicle. Further, we must ensure that a vehicle cannot deny a TRM it had sent. This implies security mechanisms have to be designed to guarantee the integrity, authenticity, and non-repudiation of TRMs in VANETs.
- 2) **Conditional Privacy:** The use of pseudonym technique is aimed at preventing unauthorized entities from discovering the true identity of a TRM generator under normal circumstances. However, in the event that a fake TRM is detected, the KGC has the ability to recover the true identity of the TRM generator.

Besides the above security and privacy challenges, our scheme for V2V communications also aims to achieve the following properties.

- 1) **Fast Response:** Traffic-related message-signature pairs received by a vehicle can be verified in a short delay.
- 2) **Certificate Management Problem Freeness:** The (anonymous) certificates used in traditional in TPBC are not required. Hence, the certificate management problem is avoided.
- 3) **Key Escrow Freeness:** Except for the vehicle itself, no other entity (e.g., RSU and KGC) has a knowledge of the

full private key of the vehicle. This implies that no entity other than the vehicle itself can sign a TRM on behalf of that vehicle.

C. Batch Verification

Batch verification is a technique used to check the validity of multiple signatures on different messages simultaneously. In VANET, it is crucial to minimize the overheads of signature verification to reduce the computational burden and improve the system's efficiency.

The most widely used batch verification methods include random subset (RS) test, small exponents (SE) test and bucket test [38]. According to [38], if the number of message-signature pairs to be verified in a batch falls within the range of 5-200, which is the typical number of message-signature pairs received by a vehicle in a batch period in VANET, the SE test will result in lower signature verification overheads than the other two methods. By exploiting the SE property, the SE test can significantly reduce the overheads of signature verification, while maintaining a high level of security.

D. Bilinear Map and Hard Problem

Our CLSS with batch verification (and hence our ES&PPS) is from bilinear map. Let $\mathbb{G}_1, \mathbb{G}_2$ be two additive groups and $\mathbb{G}_T$ be a multiplicative group of prime order q , P be a generator of $\mathbb{G}_1$, P' be a generator of $\mathbb{G}_2$, $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear map that satisfies the following properties:

- 1) **Bilinearity:** $e(aP, bP') = e(P, P')^{ab}$ for all $a, b \in \mathbb{Z}_q$.
- 2) **Non-degeneracy:** For any $P \in \mathbb{G}_1^*, P' \in \mathbb{G}_2^*$, $e(P, P') \neq 1_{\mathbb{G}_T}$.

If $\mathbb{G}_1 \neq \mathbb{G}_2$ and there is an efficiently computable isomorphism $\psi: \mathbb{G}_2 \rightarrow \mathbb{G}_1$ and $\psi(P') = P$, e is a Type 2 bilinear map; else if $\mathbb{G}_1 \neq \mathbb{G}_2$ and no such isomorphism exists, e is a Type 3 bilinear map [39]. Else if $\mathbb{G}_1 = \mathbb{G}_2$, e is a Type 1 bilinear map;

The construction of our CLSS (with batch verification) is based on Type 2 bilinear map, and the security of our CLSS (with batch verification) and our ES&PPS is based on the co-computational Diffie-Hellman (co-CDH) assumption which assumes the following problem is hard.

Definition 1 (co-CDH Problem): Given (P, aP, P', bP') , for some unknown $a, b \in \mathbb{Z}_q^*$, to compute abP .

E. Notations

Table I lists the notations commonly used in our schemes.

III. BUILDING BLOCK

In this section, we propose a CLSS with batch verification as a building block of our ES&PPS.

A. Definition of CLSS With Batch Verification

We first define the notion of CLSS which consists of the following algorithms:

- **Setup(k):** It is executed by the KGC, who generates a master secret key s and a parameter list $paramList$ based

TABLE I
NOTATION

Notation	Explanation
$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$	Cyclic groups
e	A bilinear map from $\mathbb{G}_1 \times \mathbb{G}_2$ to $\mathbb{G}_T$
P/P'	An arbitrary generator of $\mathbb{G}_1/\mathbb{G}_2$
ψ	A computable isomorphism from $\mathbb{G}_2$ to $\mathbb{G}_1$
q	The order of $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ (a large prime)
s	The master private key
P_0/P'_0	The master public key
H_0, H_1, H_2	Cryptographic hash functions
d_i/sk_i	The partial/full private key of the user whose identity is UID_i
sv_i/pk_i	The secret value/public key of the user whose identity is UID_i
$PID_{i,j}$	The j -th pseudonym of the user whose identity is UID_i
$\tilde{m}$	The number of fresh pseudonyms requested
t_i	The validity period of a pseudonym
Ts_i	The current timestamp
$E_{key}(\cdot)/D_{key}(\cdot)$	A secure symmetric key encryption scheme
$\oplus$	The XOR operation
$\parallel$	The concatenation operation
M_i	A traffic-related message (TRM)
Δ	A one time use message
D_{delay}	The average verification delay
D_{loss}	The average verification loss rate

on the security parameter k . $paramList$ is considered here to be the default input for the algorithms listed below.

- $PPKE(UID_i, s)$: It is executed by the KGC to generate the partial private key of a user. On input a user's identity UID_i and s , this algorithm outputs a partial private key d_i of the user.
- $SPRK(UID_i, d_i)$: It is executed by a user with identity UID_i to generate its full private key. On input UID_i, d_i , it first generates a secret value sv_i and then generates a full private key sk_i of the user based on sv_i and d_i .
- $SPK(UID_i, sv_i)$: It is executed by a user with identity UID_i to generate its public key. On input UID_i and sv_i , it generates a public key pk_i of the user.
- $Sign(M_i, sk_i, UID_i, pk_i)$: It is executed by a signer/user with identity UID_i , full private key sk_i and public key pk_i . On input M_i, sk_i, UID_i, pk_i , it generates a signatures σ_i on M_i .
- $Verify(M_i, \sigma_i, UID_i, pk_i)$: It is executed by a verifier. On input $M_i, \sigma_i, UID_i, pk_i$, if the signature σ_i is valid, it outputs 1; otherwise, it outputs 0.

As to a CLSS with batch verification, it additionally has a BatchVerify algorithm which is defined below:

- $BatchVerify(\{M_i, \sigma_i, UID_i, pk_i\}_{i=1}^n)$: It is executed by a verifier. Without loss of generality, here we assume (M_i, σ_i) is the message-signature pair generated by the signer whose identity is UID_i and public key is pk_i . It outputs 1 if all the signatures are valid, or 0 otherwise.

B. Our CLSS and CLSS With Batch Verification

Our concrete CLSS is as follows.

- $Setup(k)$: Given a security parameter k , the KGC generates $\langle \mathbb{G}_1, \mathbb{G}_2, \psi, \mathbb{G}_T, e, q, P, P' \rangle$ as those in Section II-D, where e is a type 2 bilinear map.

The KGC also selects $s \in \mathbb{Z}_q^*$ randomly as the master secret key and computes the master public key $P_0 = sP$, $P'_0 = sP'$, chooses three cryptographic hash functions $H_0 : \{0, 1\}^* \rightarrow \mathbb{G}_1^*$, $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$, $H_2 : \{0, 1\}^* \rightarrow \mathbb{G}_2^*$. The KGC publishes $paramList = \{\mathbb{G}_1, \mathbb{G}_2, \psi, \mathbb{G}_T, e, q, P, P', P_0, P'_0, H_0, H_1, H_2\}$.

- $PPKE(UID_i, s)$: The KGC accepts s and a user's identity $UID_i \in \{0, 1\}^*$, computes the user's partial private key $d_i = sQ_i$, where $Q_i = H_0(UID_i)$.
- $SPRK(UID_i, d_i)$: On input UID_i, d_i , the user whose identity is UID_i selects $sv_i \in \mathbb{Z}_q^*$ randomly as its secret value and sets the full private key to be $sk_i = (sv_i, d_i)$.
- $SPK(UID_i, sv_i)$: On input UID_i, sv_i , the user whose identity is UID_i sets his public key to be $pk_i = sv_i P$.
- $Sign(M_i, sk_i, UID_i, pk_i)$: On input M_i, sk_i, UID_i, pk_i , the signer whose identity is UID_i , full private key is $sk_i = (sv_i, sQ_i)$ and public key is pk_i does the following steps to generate a signature on M_i :
 - 1) Choose $r_i \in \mathbb{Z}_q^*$ randomly, set $R_i = r_i P$.
 - 2) Compute $u_i = H_1(UID_i || M_i || pk_i || R_i || \Delta)$ and $W = H_2(\Delta)$, where Δ is a one-time use message which could be generated based on a timestamp. It is assumed that the timestamp is included in M_i and Δ is generated based on the timestamp.
 - 3) Compute $V_i = u_i sv_i \psi(W) + u_i d_i + r_i(\psi(W) + P_0)$.
 - 4) Output the signature $\sigma_i = (R_i, V_i)$.
- $Verify(M_i, \sigma_i, UID_i, pk_i)$: To verify the validity of σ_i on message M_i under UID_i and pk_i , the verifier does the following:
 - 1) Compute $Q_i = H_0(UID_i)$, $u_i = H_1(UID_i || M_i || pk_i || R_i || \Delta)$, $W = H_2(\Delta)$.
 - 2) If $e(V_i, P') = e(u_i pk_i + R_i, W)e(u_i Q_i + R_i, P'_0)$, output 1; else, output 0.

In the above CLSS, if all the signers use the same Δ to generate the signatures, then these signatures could be verified using the following BatchVerify algorithm which is based on SE test. Here we assume the message-signature pair (M_i, σ_i) is generated by the signer with identity-public key pair (UID_i, pk_i) , where $i \in \{1, \dots, n\}$.

- $BatchVerify(\{M_i, \sigma_i, UID_i, pk_i\}_{i=1}^n)$: The algorithm runs as follows:
 - 1) Randomly choose $\vartheta = (\vartheta_1, \vartheta_2, \dots, \vartheta_n)$ with each $\vartheta_i \in \{0, 1\}^\ell$, where ℓ is a small integer.
 - 2) Compute $Q_i = H_0(UID_i)$, $u_i = H_1(UID_i || M_i || pk_i || R_i || \Delta)$, $W = H_2(\Delta)$.
 - 3) If $e(\sum_{i=1}^n \vartheta_i V_i, P') = e(\sum_{i=1}^n \vartheta_i (u_i pk_i + R_i), W)e(\sum_{i=1}^n \vartheta_i (u_i Q_i + R_i), P'_0)$ holds, output 1; else, output 0.

We note that the BatchVerify algorithm, combined with the existing algorithms of our CLSS, constitutes our CLSS with Batch Verification.

C. Security Model for CLSS (With Batch Verification)

According to [12], for a CLSS with batch verification, we just need to prove that the underlying CLSS is secure.

As defined in [40], a CLSS should resist the attacks from two types of adversaries. A type I adversary A_I acts as a dishonest

user who can replace any user's public key. A type II adversary A_{II} acts as a malicious KGC. He has the knowledge of the master secret key but is unable to replace the public key of the target user.

The security of a CLSS is modeled via following two games.

Game 1: This game is used to capture the attacks from a type I adversary A_I . It has three phases, and runs between a challenger C and A_I .

Setup: In this phase, C generates $paramList$, and sends $paramList$ to A_I .

Queries: In this phase, A_I can ask C the following queries in an adaptive manner.

- $PPK(UID_i)$: It allows A_I to ask the partial private key of a user whose identity is UID_i . When C receives a $PPK(UID_i)$ query, it returns the partial private key d_i of the user.
- $SVE(UID_i)$: It allows A_I to ask the secret value of a user whose identity is UID_i . When C receives a $SVE(UID_i)$ query, it returns the secret value sv_i of the user. If the user's public key has been replaced, C returns $\perp$.
- $PK(UID_i)$: It allows A_I to ask the public key of a user whose identity is UID_i . For such a query, C returns the public key pk_i of the user.
- $PKR(UID_i, pk'_i)$: When A_I makes a $PKR(UID_i, pk'_i)$ query, C replaces the public key of the user with identity UID_i to be pk'_i .
- $S(M_i, UID_i, pk_i)$: It allows A_I to ask a user's signature on a message. For such a query, C returns a signature σ_i on message M_i which is valid under UID_i and pk_i .

Forgery: Finally, A_I outputs a forgery $\{M_i^*, \sigma_i^*, UID_i^*, pk_i^*\}$. If the forgery $\{M_i^*, \sigma_i^*, UID_i^*, pk_i^*\}$ satisfy the following conditions, we say A_I wins Game 1:

- 1) σ_i^* is a valid signature on message M_i^* under the user's identity UID_i^* and public key pk_i^* .
- 2) A_I has never asked the query $PPK(UID_i^*)$.
- 3) $S(M_i^*, UID_i^*, pk_i^*)$ has never been queried.

Game 2: This game is used to capture the attacks from a type II adversary A_{II} . It has three phases, and runs between a challenger C and A_{II} .

Setup: C generates the system parameter list $paramList$ and the system's master secret key s . Then C sends $paramList$ and s to A_{II} .

Queries: A_{II} can ask C the queries as those defined in game 1 in an adaptive manner.

Forgery: Finally, A_{II} outputs a forgery $\{M_i^*, \sigma_i^*, UID_i^*, pk_i^*\}$.

If the forgery $\{M_i^*, \sigma_i^*, UID_i^*, pk_i^*\}$ satisfy the following conditions, we say A_{II} wins Game 2:

- 1) σ_i^* is a valid signature on M_i^* under the user's identity UID_i^* and public key pk_i^* .
- 2) A_{II} has never requested queries $SVE(UID_i^*)$ and $PKR(UID_i^*, pk_i^*)$.
- 3) $S(M_i^*, UID_i^*, pk_i^*)$ has never been queried.

Definition 2: We say that a CLSS is secure, i.e., the scheme is existentially unforgeable against chosen message attacks, if the probability for A_I/A_{II} to win Game 1/2 is negligible.

For the formal security proof of our CLSS, please refer to Section VI.

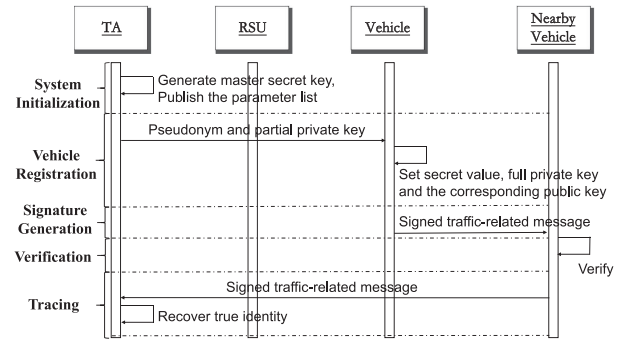


Fig. 2. Overview of our scheme.

IV. THE PROPOSED SCHEME

We present our ES&PPS in this section.

A. High Level Description

As shown in Fig. 2, our ES&PPS is composed of five phases: System Initialization, Vehicle Registration, Signature Generation, Verification, and Tracing.

- **System Initialization:** The KGC generates the system master secret key and publishes parameter list in this phase.
- **Vehicle Registration:** In this phase, each vehicle is required to be registered with the KGC. Particularly, the KGC generates a partial private key and pseudonyms for each vehicle, and sends them to each vehicle via a secure channel. When a vehicle receives its partial private key and pseudonyms, it generates its full private keys and public keys.
- **Signature Generation:** The vehicles in VANETs need to broadcast TRMs periodically. In this phase, each vehicle first signs a TRM generated by itself using its own full private key, and then broadcasts the signed TRM to the nearby vehicles.
- **Verification:** The vehicles may receive many signed TRMs within a very short period. During this phase, the batch verification technique is employed to verify the validity of the signed TRMs.
- **Tracing:** When a fake TRM is found, the KGC has the ability to recover the true identity of the generator.

Next, we show each phase in detail.

B. System Initialization

In this phase, the system master secret key and parameter list are generated as follows.

- 1) Given a security parameter k , run the algorithm $Setup(k)$ of our CLSS to produce master secret key s and $paramList = \{\mathbb{G}_1, \mathbb{G}_2, \psi, \mathbb{G}_T, e, q, P, P', P_0, P'_0, H_0, H_1, H_2\}$.
- 2) Choose a secure symmetric encryption scheme $E_{key}(\cdot)/D_{key}(\cdot)$ and a secret key key of $E_{key}(\cdot)/D_{key}(\cdot)$.
- 3) Set the system master secret key to be (key, s) .
- 4) Set the parameter list to be $paramList' = \{paramList, E_{key}(\cdot)/D_{key}(\cdot)\}$ and publish $paramList'$.

C. Vehicle Registration

Before a vehicle joins the system, it needs to be registered with the KGC. Assume the vehicle is V_i . The detailed processes are as follows:

- 1) V_i first submits a registration request $(UID_i, \tilde{m})$ to the KGC via a secure channel, where UID_i is the true identity of the vehicle and the $\tilde{m}$ is the number of fresh pseudonyms requested.
- 2) Once a registration request from V_i is received, the KGC performs the following steps.
 - a) Generate a pool of pseudonyms by computing $PID_{i,j} = E_{key}(UID_i || j) || t_i$ for $j \in \{1, \dots, \tilde{m}\}$, where t_i is the validity period of a pseudonym.
 - b) For $j \in \{1, \dots, \tilde{m}\}$, run $PPKE(PID_{i,j}, s)$ of our CLSS to obtain the partial private key $d_{i,j}$ corresponding to $PID_{i,j}$.
 - c) Send $\{(PID_{i,j}, d_{i,j})\}_{j=1}^{\tilde{m}}$ to V_i via the secure channel.
- 3) On receiving $\{(PID_{i,j}, d_{i,j})\}_{j=1}^{\tilde{m}}$, for $j \in \{1, \dots, \tilde{m}\}$, V_i first runs $SPRK(PID_{i,j}, d_{i,j})$ of our CLSS to generate the corresponding full private key $sk_{i,j} = (sv_{i,j}, d_{i,j})$ and then runs $SPK(PID_{i,j}, sv_{i,j})$ of our CLSS to generate the corresponding public key $pk_{i,j}$.

D. Signature Generation

Before V_i broadcasts a TRM generated by itself, it has to sign TRM at first.

Here we assume that V_i needs to broadcast a TRM M_i which contains a timestamp Ts_i and the current pseudonym/full private key/public key of V_i is $PID_{i,j}/sk_{i,j}/pk_{i,j}$. V_i first runs the algorithm $Sign(M_i, sk_{i,j}, PID_{i,j}, pk_{i,j})$ of our CLSS to produce the signature $\sigma_{i,j}$ on M_i . Then V_i broadcasts the signed TRM $\{PID_{i,j}, pk_{i,j}, M_i, \sigma_{i,j}\}$.

E. Verification

A vehicle may receive a large number of TRMs in a short period of time. The signed TRMs generated using the same one-time use message can be verified in a batch. We note that, one may use the technique in [8] to generate such a one-time use message such that the signed TRMs verified in a batch corresponding to the same one-time use message.

Without loss of generality, we assume the one-time use message is Δ , the signed TRMs received by a vehicle are $\{(PID_{1,j}, pk_{1,j}, M_1, \sigma_{1,j}), \dots, (PID_{n,j}, pk_{n,j}, M_n, \sigma_{n,j})\}$. The vehicle runs the algorithm $BatchVerify(\{M_i, \sigma_{i,j}, PID_{i,j}, pk_{i,j}\}_{i=1}^n)$ of our CLSS to verify the validity of these signed TRMs. If the algorithm outputs 1, these TRMs are accepted.

F. Tracing

There may be some malicious vehicles in VANETs. They may broadcast TRMs to mislead the other vehicles. If a fake signed TRM is found by a vehicle, it may ask the KGC to recover the true identity of the generator.

Suppose the fake signed TRM is $(PID_{i,j}, pk_{i,j}, M_i, \sigma_{i,j})$, where $\sigma_{i,j}$ is a valid signature on M_i under $PID_{i,j}, pk_{i,j}$. Once the KGC receives this fake signed TRM, the

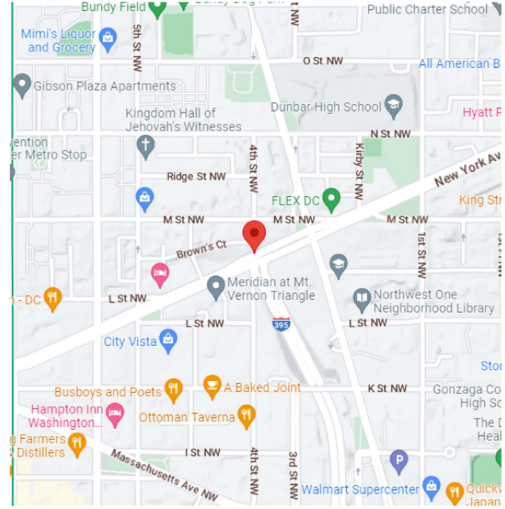


Fig. 3. The map of simulation scenario.

KGC first gets $UID_i || j$ by computing $D_{key}(PID_{i,j}) = D_{key}(E_{key}(UID_i || j))$, then the KGC can get the true identity UID_i of the corresponding generator.

V. PERFORMANCE EVALUATION

To show the efficiency of our ES&PPS, we performed several simulations. Further, we conducted a comparison of the computational and transmission overheads of our scheme with those of the recent schemes.

A. Simulation

Our simulations were conducted on an Ubuntu machine with the following specifications: an Intel Core i7-4790 processor running at 3.6 GHz. The machine ran the Ubuntu operating system and was equipped with NS-3, VanetMobisim, and the MIRACL cryptographic library. We chose to use a BN curve with a 256-bit q [41]. In addition, we selected small exponents of size 60 bits for our simulations [38]. Our simulations were conducted on a road scenario depicted in Fig. 3, covering an area of $1.0 \times 1.0 \text{ km}^2$. The vehicle in our simulations traveled at speeds ranging from 50 km/h to 60 km/h, with a corresponding vehicle density (i.e., number of vehicles per square kilometer) varying from 50/km² to 300/km². The maximum communication range of each vehicle was set to 300 m, and the channel bandwidth was set to 6Mbps. In terms of message size, each signature comprised approximately 64 bytes, while each message broadcast by a vehicle consisted of 252 bytes. The duration of each simulation run was 100 s.

Our performance metrics include the average verification delay D_{delay} and the average verification loss rate D_{loss} . To establish a clear understanding of these metrics, we first define the related notations. We define $\mathbb{S}$ as the simulation area, $N_{\mathbb{S}}$ as the total number of vehicles within $\mathbb{S}$, d as the batch period (i.e., the duration from the start to the end of receiving messages before batch verification), $n_d^{\max}$ as the maximum number of signed TRMs that can be verified by a vehicle in d , n_d^v as the

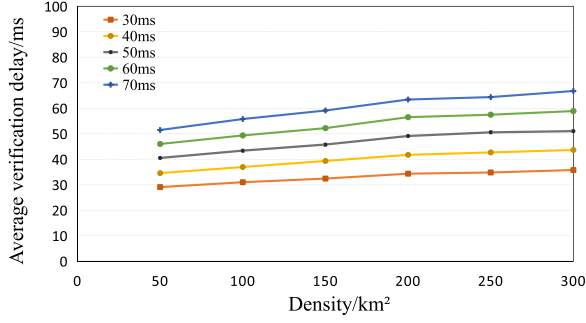


Fig. 4. Average verification delay.

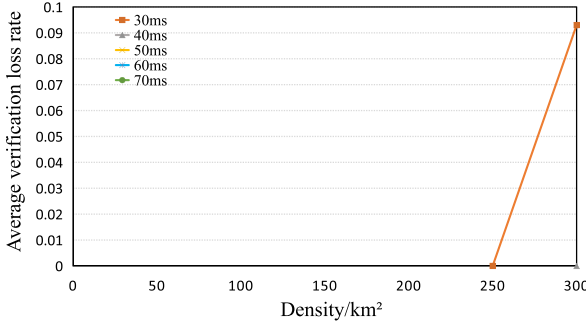


Fig. 5. Average verification loss rate.

total number of signed TRMs received by a vehicle v in d , Δ_d as the average waiting time of a signed TRM to be verified before batch verification, and $T_{ver}^{N_d^v}$ as the time required to verify the signed TRMs (which should not exceed $n_d^{\max}$) received by v in d .

The average verification delay D_{delay} is defined as the average time cost from sending a signed TRM by a vehicle to its verification by another vehicle. Therefore, we have

$$D_{delay} = \Delta_d + \frac{1}{N_S} \sum_{v \in S} T_{ver}^{N_d^v}. \quad (1)$$

Fig. 4 illustrates the relationship among average verification delay, batch period d and vehicle density. Fig. 4 shows that D_{delay} increases with both batch period and vehicle density. However, it is important to note that D_{delay} always remains below 100 ms, which is the maximum allowable delay according to [42].

The average verification loss rate D_{loss} refers to the average probability that a signed TRM cannot be verified before the next batch verification. Therefore we define the average verification loss rate D_{loss} as:

$$D_{loss} = 1 - \frac{1}{N_S} \sum_{v \in S} \frac{n_d^{\max}}{n_d^v}. \quad (2)$$

Fig. 5 illustrates the relationship among average verification loss rate, batch period d , and vehicle density. Obviously, when the batch period is set to 40 ms, the average verification loss rate approaches 0 even when the vehicle density reaches 300/km².

TABLE II
COMPARISON OF VERIFICATION OVERHEADS

Scheme	Verification overhead	Type of bilinear map	Secure
[24]	$5T_b + 2nT_{m1} + (2n + 3)T_H$	Type 1	Yes
[30]	$4T_b + 2nT_{m1}$	Type 1	No
[31]	$4T_b + 2nT_{m1}$	Type 1	Unknown
[33]	$(n + 1)T_m$	Pairing Free	No
[32]	$(2n + 1)T_m$	Pairing Free	Unknown
[35]	$3T_b + 2nT_{m1} + nT_H + 2nT_{s1}$	Type 1	Yes
[36]	$(n + 1)T_m$	Pairing Free	No
[37]	$(2n + 2)T_m$	Pairing Free	Unknown
Our scheme	$3T_b + 2nT_{m1} + nT_H + 3nT_{s1}$	Type 2	Yes

Given that the average verification delay should be minimized, we recommend setting the batch period to 40ms.

B. Comparison

The efficiency of our ES&PPS is mainly dominated by our proposed CLSS with batch verification. To evaluate its efficiency, we compare our CLSS with batch verification to the CLSS with batch verification presented in [35], [36], [37]. Additionally, since certificateless aggregate signature schemes serve a similar purpose as CLSS with batch verification, we compare our CLSS with batch verification to the most efficient and provably secure certificateless aggregate signature scheme in [24] and the most recently proposed certificateless aggregate signature schemes in [30], [31], [32], [33].

Table II compares the signature verification overheads of our scheme with the schemes in [24], [30], [31], [32], [33], [35], [36], [37]. In the table, we only consider costly operations. T_b , T_{m1} , T_m , T_H , and T_{s1} denote the time required to compute a bilinear map operation, a scale multiplication operation in $\mathbb{G}_1$, a scale multiplication operation on elliptic curve cryptography (ECC), a hash to group operation, and a small exponents test operation in $\mathbb{G}_1$, respectively. We note that the schemes in [24], [30], [31], [35] are designed from Type 1 bilinear map and our scheme is designed from Type 2 bilinear map. As showed in [43], a Type 1 bilinear map operation is much more expensive than a Type 2 bilinear map operation. Therefore, when a scheme is designed using a Type 1 bilinear map, the values of T_b , T_{m1} , T_H , and T_{s1} are significantly higher than those of a scheme designed using a Type 2 bilinear map.

Fig. 6 compares the simulation results of the verification overheads of the schemes in [24], [30], [31], [35] (using an SSP curve with a 256-bit q), the schemes in [32], [33], [36], [37] (using a BN curve with a 256-bit q) with the overhead of our scheme (using a BN curve with a 256-bit q). As shown in the figure, with the number of messages increasing from 10 to 100, the time costs of verification for [24], [30], [31], [32], [33], [35], [36], [37] and our scheme range from 183 ms to 1200 ms, 80 ms to 337 ms, 80 ms to 337 ms, 1 ms to 11 ms, 2 ms to 23 ms, 125 ms to 912 ms, 1 ms to 11 ms, 2 ms to 23 ms and 16 ms

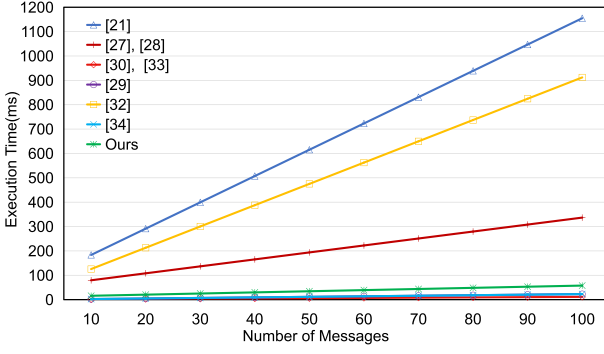


Fig. 6. Comparison of verification overheads.

TABLE III
COMPARISON OF TRANSMISSION OVERHEADS

Scheme	Signature length	Public key length
[24]	$2l_{\mathbb{G}_1^{ssp}}$	$l_{\mathbb{G}_1^{ssp}}$
[30]	$2l_{\mathbb{G}_1^{ssp}}$	$l_{\mathbb{G}_1^{ssp}}$
[31]	$2l_{\mathbb{G}_1^{ssp}}$	$l_{\mathbb{G}_1^{ssp}}$
[33]	$3l_{\mathbb{G}^{ecc}} + l_{\mathbb{Z}_p^*} + 4$	$l_{\mathbb{G}^{ecc}}$
[32]	$l_{\mathbb{G}^{ecc}} + l_{\mathbb{Z}_p^*}$	$l_{\mathbb{G}^{ecc}}$
[35]	$2l_{\mathbb{G}_1^{ssp}}$	$l_{\mathbb{G}_1^{ssp}}$
[36]	$2l_{\mathbb{G}^{ecc}}$	$l_{\mathbb{G}^{ecc}}$
[37]	$2l_{\mathbb{G}^{ecc}}$	$l_{\mathbb{G}^{ecc}}$
Our scheme	$2l_{\mathbb{G}_1^{bn}}$	$l_{\mathbb{G}_1^{bn}}$

to 58 ms, respectively. The results indicate that the BatchVerify algorithm of our ES&PPS is more efficient for verifying a batch of signatures compared to [24], [30], [31], [35]. The schemes in [32], [33], [36], [37] are only slightly more efficient than ours. However, they are not secure or without a formal security proof.

Table III presents a comparison of the transmission overheads of our scheme with those of the schemes in [24], [30], [31], [32], [33], [35], [36], [37]. In the table, $l_{\mathbb{Z}_p^*}/l_{\mathbb{G}_1^{ssp}}/l_{\mathbb{G}_1^{bn}}/l_{\mathbb{G}^{ecc}}$ indicates the bit-length of an element in $\mathbb{Z}_p^*/\mathbb{G}_1$ corresponding to a SSP curve/ $\mathbb{G}_1$ corresponding to a BN curve/a group corresponding to a BN curve. According to our simulations, $l_{\mathbb{Z}_p^*}$, $l_{\mathbb{G}_1^{ssp}}$, $l_{\mathbb{G}_1^{bn}}$, $l_{\mathbb{G}^{ecc}}$ are 32 bytes, 192 bytes, 32 bytes, 32 bytes respectively. It is clear from the table that our scheme has the same or even lower transmission overheads in terms of the signature and public key length compared to the other schemes in [24], [30], [31], [32], [33], [35], [36], [37].

VI. SECURITY ANALYSIS

We first prove that our CLSS (with batch verification) is secure, i.e., the scheme is secure against both type I and II adversaries, then we show our ES&PPS satisfies the properties of message authentication and conditional privacy defined in Section II-B.

Theorem 1: Assume a scale multiplication operation in $\mathbb{G}_1$ takes time T_{m1} , a scale multiplication operation in $\mathbb{G}_2$

takes time T_{m2} and all other operations take unit time $\bar{t}$. If there exists A_I with advantage ε to forge a signature of our CLSS scheme in an attack modeled by Game 1 within a time limit of t , after querying at most q_{H_0} times H_0 queries, q_{H_1} times H_1 queries, q_{H_2} times H_2 queries, q_{ppk} times $PPK(UID_i)$ queries, q_{pk} times $PK(UID_i)$ queries, and q_s times $S(M_i, UID_i, pk_i)$ queries, then C can use A_I to solve the co-CDH problem with time $t' \approx 2t + (2q_{H_0} + 2q_{pk} + 2q_{ppk} + 10q_s + 3)T_{m1} + 2q_{H_2}T_{m2} + O(q_{H_1} + q_s + 1)\bar{t}$ and with advantage $\xi \geq \frac{\varepsilon}{e(q_{ppk}+1)} \left[\frac{\varepsilon}{e(q_{ppk}+1)(q_{H_1}+q_s+1)} - \frac{1}{q} \right]$.

Proof: Suppose C is given a co-CDH instance (P, P', bP, aP') . We show how the co-CDH problem can be solved by C .

Initialization: C sets $P'_0 = aP'$, $P_0 = \psi(P'_0)$, then gives $paramList = \{\mathbb{G}_1, \mathbb{G}_2, \psi, \mathbb{G}_T, e, q, P, P', P_0, P'_0, H_0, H_1, H_2\}$ to A_I . C also maintains several lists which will be defined below.

Queries: C answers A_I 's queries as follows.

H_0 queries: C keeps an initially empty list L_{H_0} which stores tuples of the form $(UID_i, c_i, \alpha_i, Q_i)$. When C receives an $H_0(UID_i)$ query, it first checks whether the tuple $(UID_i, c_i, \alpha_i, Q_i)$ is in L_{H_0} . If it does, the same value will be returned; otherwise, C flips a coin $c_i \in \{0, 1\}$ with probability ζ of getting 0 and probability $1 - \zeta$ of getting 1, then does the following:

- 1) If $c_i = 1$, pick $\alpha_i \in \mathbb{Z}_q^*$ randomly, compute $Q_i = \alpha_i bP$, then add $(UID_i, c_i, \alpha_i, Q_i)$ to L_{H_0} and return Q_i .
- 2) Else, pick $\alpha_i \in \mathbb{Z}_q^*$ randomly, compute $Q_i = \alpha_i P$, then add $(UID_i, c_i, \alpha_i, Q_i)$ to L_{H_0} and return Q_i .

H_1 queries: C keeps an initially empty list L_{H_1} which stores tuples of the form $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$. When C receives an $H_1(UID_i, M_i, pk_i, R_i, \Delta_i)$ query, the same value will be returned if the corresponding tuple $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ already exists in L_{H_1} ; otherwise, C picks $u_i \in \mathbb{Z}_q^*$ randomly, adds $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ to L_{H_1} and returns u_i .

H_2 queries: C keeps an initially empty list L_{H_2} which stores tuples of the form $(\gamma_i, \Delta_i, W_i)$. When C receives an $H_2(\Delta_i)$ query, the same value will be returned if the corresponding tuple $(\gamma_i, \Delta_i, W_i)$ already exists in L_{H_2} ; otherwise, C picks $\gamma_i \in \mathbb{Z}_q^*$ randomly, computes $W_i = \gamma_i P'$, adds $(\gamma_i, \Delta_i, W_i)$ to L_{H_2} and returns W_i .

$PK(UID_i)$ queries: C keeps an initially empty list L which stores tuples of the form (UID_i, d_i, sv_i, pk_i) . When C receives a $PK(UID_i)$ query, it does the following:

- 1) If (UID_i, d_i, sv_i, pk_i) appears in L and $pk_i \neq \perp$, return pk_i .
- 2) Else if (UID_i, d_i, sv_i, pk_i) appears in L and $pk_i = \perp$, pick a random $sv_i \in \mathbb{Z}_q^*$, set $pk_i = sv_i P$ and return pk_i .
- 3) Else pick a random $sv_i \in \mathbb{Z}_q^*$, set $pk_i = sv_i P$ and $d_i = \perp$, then add (UID_i, d_i, sv_i, pk_i) to L and return pk_i .

$PPK(UID_i)$ queries: When C receives a query $PPK(UID_i)$, the same value will be return if the tuple (UID_i, d_i, sv_i, pk_i) appears in L ; otherwise, C first makes an $H_0(UID_i)$ query to obtain $(UID_i, c_i, \alpha_i, Q_i)$, then does the following:

- 1) If $c_i = 1$, abort.
- 2) Else if (UID_i, d_i, sv_i, pk_i) in L and $d_i = \perp$, set $d_i = \alpha_i P_0$, return d_i .
- 3) Else, set $d_i = \alpha_i P_0$, $sv_i = \perp$, $pk_i = \perp$, return d_i and add (UID_i, d_i, sv_i, pk_i) to L .

PKR (UID_i, pk'_i) queries: When C receives a **PKR** (UID_i, pk'_i) query, it first checks whether the tuple (UID_i, d_i, sv_i, pk_i) is in L . If it does, C sets $pk_i = pk'_i$. Otherwise, C makes a **PK** (UID_i) query to obtain the tuple (UID_i, d_i, sv_i, pk_i) , then sets $pk_i = pk'_i$. Note that A_I need not to provide the secret value that was used to generate pk'_i .

SVE (UID_i) queries: When C receives a **SVE** (UID_i) query, it does the following:

- 1) If (UID_i, d_i, sv_i, pk_i) appears in the list L and $pk_i \neq \perp$, return sv_i .
- 2) If (UID_i, d_i, sv_i, pk_i) appears in the list L and $pk_i = \perp$, make a query **PK** (UID_i) and return sv_i .
- 3) Else make a query **PK** (UID_i) to generate the tuple (UID_i, d_i, sv_i, pk_i) and return sv_i .

We note that if the public key of a user whose identity is UID_i is replaced, the initial secret value is returned, since A_I need not to provide the secret value corresponding to the replaced public key.

S (M_i, UID_i, pk_i) queries: When C receives a **S** (M_i, UID_i, pk_i) query, it first generates Δ_i based on the timestamp included in M_i , then makes $H_0(UID_i)$ and $H_2(\Delta_i)$ queries to obtain $(UID_i, c_i, \alpha_i, Q_i)$ and $(\gamma_i, \Delta_i, W_i)$, then does the following:

- 1) If $c_i = 1$, do the following:
 - a) Choose $r_i, u_i \in \mathbb{Z}_q^*$ randomly, set $R_i = r_i P - u_i Q_i$. If the tuple $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ appears in the list L_{H_1} , redo this step; else add $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ to L_{H_1} .
 - b) Compute $V_i = \gamma_i u_i pk_i + \gamma_i R_i + r_i P_0$.
 - c) Output $\sigma_i = (R_i, V_i)$.
- 2) If $c_i = 0$, do the following:
 - 1) Choose $r_i \in \mathbb{Z}_q^*$ at random, sets $R_i = r_i P$.
 - 2) Make an $H_1(UID_i, M_i, pk_i, R_i, \Delta_i)$ query to obtain u_i .
 - 3) Compute $V_i = \gamma_i u_i pk_i + \gamma_i R_i + u_i \alpha_i P_0 + r_i P_0$.
 - 4) Return $\sigma_i = (R_i, V_i)$.

Forgery: Finally, A_I outputs a forgery $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$. If the corresponding tuple $(UID_i^*, M_i^*, R_i^*, pk_i^*, u_i^*)$ does not appear in L_{H_1} , C would add the tuple $(UID_i^*, M_i^*, R_i^*, pk_i^*, u_i^*)$ to the L_{H_1} . In addition, it requires that σ_i^* is a valid signature on M_i^* under UID_i^* and pk_i^* , and $c_i^* = 1$, where c_i^* is corresponding to a tuple $(UID_i^*, c_i^*, \alpha_i^*, Q_i^*)$ in L_{H_0} ; otherwise, C aborts.

Solving the co-CDH Problem: Since A_I cannot output a valid forgery if he hasn't queried C about $H_2(\Delta_i^*)$, $H_0(UID_i^*)$, **PK** (UID_i^*) , C can recover the tuples $(\gamma_i^*, \Delta_i^*, W_i^*)$ from list L_{H_2} , $(UID_i^*, c_i^*, \alpha_i^*, Q_i^*)$ from list L_{H_0} , and $(UID_i^*, d_i^*, sv_i^*, pk_i^*)$ from list L corresponding to A_I 's forgery $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$. By replays of C with the same random tape but different choices of H_1 , as done in the forking lemma [44], we get signatures $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$ and $\{M_i^*, \sigma_i^{*'} = (R_i^*, V_i^{*'}), UID_i^*, pk_i^*\}$ which are expected to be valid

ones with regard to hash function H_1 and H_1' having different values $u_i^* \neq u_i^{*'}$ on $(UID_i^*, M_i^*, pk_i^*, R_i^*, \Delta_i^*)$, respectively. We observe that the signatures must satisfy $e(V_i^*, P') = e(u_i^* pk_i^* + R_i^*, W_i^*) e(u_i^* Q_i^* + R_i^*, P'_0)$ and $e(V_i^{*'}, P') = e(u_i^{*' } pk_i^* + R_i^*, W_i^*) e(u_i^{*' } Q_i^* + R_i^*, P'_0)$ respectively. By our setting, since $P'_0 = aP$, $Q_i^* = \alpha_i^* bP$, C can compute $abP = \alpha_i^{*-1} [(u_i^* - u_i^{*' })^{-1} (V_i^* - V_i^{*' }) - \gamma_i^* pk_i^*]$, which is the solution of co-CDH problem.

Next, we calculate ξ which is the probability of C successfully solving the co-CDH problem. Let acc be the probability that C does not abort in the above simulation. According to our simulation, this probability is determined by the following events:

- Event1: C does not abort as a result of any of A_I 's queries above.
- Event2: σ_i^* is a valid and nontrivial forgery.
- Event3: Event2 occurs and $c_i^* = 1$.

It is easy to see that $acc = \Pr[Event1 \wedge Event2 \wedge Event3] \geq \zeta^{q_{ppk}} (1 - \zeta) \varepsilon$. We can find the value of ζ that maximize this probability is $\zeta = \frac{q_{ppk}}{1 + q_{ppk}}$ and the maximized probability is $\frac{\varepsilon}{e(1 + q_{ppk})}$. According to [44], we have

$$\begin{aligned} \xi &\geq acc \left(\frac{acc}{q_{H_1} + q_s + 1} - \frac{1}{q} \right) \\ &\geq \frac{\varepsilon}{e(q_{ppk} + 1)} \left[\frac{\varepsilon}{e(q_{ppk+1})(q_{H_1} + q_s + 1)} - \frac{1}{q} \right]. \end{aligned} \quad (3)$$

The running time t' required for C to solve the co-CDH problem can be approximated as twice the time it takes for C to simulate without aborting, plus the time required to compute the co-CDH solution. The time it takes for C to simulate without aborting can be calculated as $t + (q_{H_0} + q_{pk} + q_{ppk} + 5q_s)T_{m1} + q_{H_2}T_{m2} + O(q_{H_1} + q_s + 1)\bar{t}$. The time it takes for C to solve the co-CDH problem can be calculated as $3T_{m1}$. Therefore, we have $t' \approx 2t + (2q_{H_0} + 2q_{pk} + 2q_{ppk} + 10q_s + 3)T_{m1} + 2q_{H_2}T_{m2} + O(q_{H_1} + q_s + 1)\bar{t}$.

Theorem 2: If there exists A_{II} with advantage ε to forge a signature of our CLSS scheme in an attack modeled by Game 2 within a time limit of t , after querying at most q_{H_0} times H_0 queries, q_{H_1} times H_1 queries, q_{H_2} times H_2 queries, q_{pk} times **PK** (UID_i) queries, q_{sv} times **SVE** (UID_i) queries, and q_s times **S** (M_i, UID_i, pk_i) queries, then C can use A_{II} to solve the co-CDH with time $t' \approx 2t + (2q_{H_0} + 2q_{pk} + 10q_s + 3)T_{m1} + 2q_{H_2}T_{m2} + O(q_{H_1} + q_s + 1)\bar{t}$ and with advantage $\frac{4\varepsilon}{e(q_{sv}+2)^2} \left[\frac{4\varepsilon}{e(q_{sv}+2)^2(q_{H_1}+q_s+1)} - \frac{1}{q} \right]$.

Proof: Suppose C is given a co-CDH instance (P, P', bP, aP') . We show how the co-CDH problem can be solved.

Initialization: C picks $s \in \mathbb{Z}_q^*$ as the master secret key, sets $P_0 = sP, P'_0 = sP'$. Then C gives the *paramList* = $\{\mathbb{G}_1, \mathbb{G}_2, \psi, \mathbb{G}_T, e, q, P, P', P_0, P'_0, H_0, H_1, H_2\}$ and the master secret key s to A_{II} . C also initializes several empty lists which will be define below. Since the A_{II} knows the master secret key s , he can do **PPK** (UID_i) himself.

Queries: C answers A_{II} 's queries as follows.

H_0 queries: C keeps an initially empty list L_{H_0} which stores tuples of the form (UID_i, α_i, Q_i) . When C receives an $H_0(UID_i)$ query, it first checks whether the tuple (UID_i, α_i, Q_i) is in L_{H_0} . If it does, the same value will be returned; otherwise, C picks $\alpha_i \in \mathbb{Z}_q^*$ randomly, computes $Q_i = \alpha_i P$, adds (UID_i, α_i, Q_i) to L_{H_0} and returns Q_i .

H_1 queries: C keeps an initially empty list L_{H_1} which stores tuples of the form $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$. When C receives an $H_1(UID_i, M_i, pk_i, R_i, \Delta_i)$ query, the same value will be returned if the corresponding tuple $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ already exists in the L_{H_1} ; otherwise, C picks $u_i \in \mathbb{Z}_q^*$ randomly, adds $(UID_i, M_i, pk_i, R_i, \Delta_i, u_i)$ to L_{H_1} and returns u_i .

H_2 queries: C keeps an initially empty list L_{H_2} which stores tuples of the form $(\beta_i, c_i, \Delta_i, W_i)$. When C receives an $H_2(\Delta_i)$ query, the same value will be returned if the corresponding tuple $(\beta_i, c_i, \Delta_i, W_i)$ already exists in L_{H_2} ; otherwise, C first flips a coin $c_i \in \{0, 1\}$ with probability ζ of getting 0 and probability $1 - \zeta$ of getting 1, then does the follows:

- 1) If $c_i = 1$, select a random $\beta_i \in \mathbb{Z}_q^*$, compute $W_i = \beta_i aP'$, add $(\beta_i, c_i, \Delta_i, W_i)$ to L_{H_2} and return W_i .
- 2) Else, select $\beta_i \in \mathbb{Z}_q^*$ randomly, compute $W_i = \beta_i P'$, add $(\beta_i, c_i, \Delta_i, W_i)$ to L_{H_2} and return W_i .

$PK(UID_i)$ queries: C keeps an initially empty list L which stores tuples of the form $(UID_i, d_i, pk_i, sv_i, f_i)$. When C receives a $PK(UID_i)$ query, C does the following:

- 1) If $(UID_i, d_i, pk_i, sv_i, f_i)$ appears in L and $pk_i \neq \perp$, return pk_i .
- 2) Else, first flip a coin $f_i \in \{0, 1\}$ with probability ζ of getting 0 and probability $1 - \zeta$ of getting 1, then does the follows:
 - a) If $f_i = 1$, pick $sv_i \in \mathbb{Z}_q^*$ randomly, set $pk_i = sv_i bP$, $d_i = \perp$, add $(UID_i, d_i, pk_i, sv_i, f_i)$ to L , return pk_i .
 - b) Else, pick $sv_i \in \mathbb{Z}_q^*$ randomly, set $pk_i = sv_i P$ and $d_i = \perp$, then add $(UID_i, d_i, pk_i, sv_i, f_i)$ to L , return pk_i .

$PKR(UID_i, pk_i')$ queries: C does the following when a $PKR(UID_i, pk_i')$ query is received:

- 1) If the tuple $(UID_i, d_i, pk_i, sv_i, f_i)$ appears in L , do the following:
 - a) If $f_i = 0$, set $pk_i = pk_i'$.
 - b) Else, abort.
- 2) Else, make a $PK(UID_i)$ to generate the tuple $(UID_i, d_i, pk_i, sv_i, f_i)$. If $f_i = 0$, C sets $pk_i = pk_i'$, else abort.

$SVE(UID_i)$ queries: When A_{II} makes a $SVE(UID_i)$ query, C first makes a query $PK(UID_i)$ and recovers the tuple $(UID_i, d_i, pk_i, sv_i, f_i)$ from L . Then, if $f_i = 0$, C returns the sv_i ; else, aborts. We observe that if the public key of a user whose identity is UID_i is replaced, the initial secret value is returned.

$S(M_i, UID_i, pk_i)$ queries: When A_{II} makes a $S(M_i, UID_i, pk_i)$ query, C first generates Δ_i based on the timestamp included in M_i , then makes an $H_0(UID_i)$ query, an $H_2(\Delta_i)$ query and a $PK(UID_i)$ query to get the tuples (UID_i, α_i, Q_i) , $(\beta_i, c_i, \Delta_i, W_i)$ and $(UID_i, d_i, pk_i, sv_i, f_i)$. C then does the following:

- 1) If $f_i = 1$ and $c_i = 1$, do the following:

- a) Choose $r_i, u_i \in \mathbb{Z}_q^*$ randomly, set $R_i = r_i P - u_i pk_i$. If the tuple $(UID_i, M_i, R_i, pk_i, u_i)$ appears in L_{H_1} , redo this step; else, add $(UID_i, M_i, R_i, pk_i, u_i)$ to L_{H_1} .
 - b) Compute $V_i = r_i \psi(W_i) + u_i sQ_i + r_i P_0$.
 - c) Return $\sigma_i = (R_i, V_i)$.
- 2) Else, do the following:
 - a) Choose $r_i \in \mathbb{Z}_q^*$ randomly, set $R_i = r_i P$.
 - b) Make an $H_1(UID_i, M_i, pk_i, R_i, \Delta_i)$ query to obtain u_i .
 - c) Compute $V_i = \beta_i u_i pk_i + \beta_i R_i + u_i sQ_i + r_i P_0$.
 - d) Return $\sigma_i = (R_i, V_i)$.

Forgery: A_{II} finally outputs a forgery $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$. If the corresponding tuple $(UID_i^*, M_i^*, R_i^*, pk_i^*, u_i^*)$ does not appear in L_{H_1} , C would add the tuple $(UID_i^*, M_i^*, R_i^*, pk_i^*, u_i^*)$ to the L_{H_1} . In addition, it requires that σ_i^* is a valid signature on M_i^* under UID_i^* and pk_i^* , $c_i^* = 1$, $f_i^* = 1$, where c_i^* is corresponding to a tuple $(\beta_i^*, c_i^*, \Delta_i^*, W_i^*)$ in L_{H_2} , and f_i^* is corresponding to a tuple $(UID_i^*, d_i^*, pk_i^*, sv_i^*, f_i^*)$ in L ; otherwise, C aborts.

Solving the co-CDH Problem: Similar to the proof of the Theorem 1, C first recovers the tuples $(UID_i^*, \alpha_i^*, Q_i^*)$ from list L_{H_0} , $(\beta_i^*, c_i^*, \Delta_i^*, W_i^*)$ from list L_{H_2} and $(UID_i^*, d_i^*, pk_i^*, sv_i^*, f_i^*)$ from list L corresponding to A_{II} 's forgery $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$. By replays of C with the same random tape but different choices of H_1 , as done in the forking lemma [44], we get signatures $\{M_i^*, \sigma_i^* = (R_i^*, V_i^*), UID_i^*, pk_i^*\}$ and $\{M_i^*, \sigma_i^{*'} = (R_i^*, V_i^{*'}), UID_i^*, pk_i^*\}$ which are expected to be valid ones with regard to hash function H_1 and H_1' having different values $u_i^* \neq u_i^{*'}$ on $(UID_i^* || M_i^* || pk_i^* || R_i^* || \Delta_i^*)$, respectively. We observe that the signatures must satisfy $e(V_i^*, P') = e(u_i^* pk_i^* + R_i^*, W_i^*) e(u_i^* Q_i^* + R_i^*, P_0)$ and $e(V_i^{*'}, P') = e(u_i^{*' } pk_i^* + R_i^*, W_i^*) e(u_i^{*' } Q_i^* + R_i^*, P_0)$ respectively. By our setting, since $W_i^* = \beta_i^* aP'$, $pk_i^* = sv_i^* bP$, C can compute $abP = sv_i^{*-1} \beta_i^{*-1} [(u_i^* - u_i^{*' })^{-1} (V_i^* - V_i^{*' }) - sQ_i^*]$, which is the solution of co-CDH problem.

Next, we calculate the probability ξ which is the probability for C to solve the co-CDH problem. Let acc be the probability that C does not abort in the above simulation. According to our simulation, this probability is determined by the following events:

Event1: C does not abort as a result of any of A_{II} 's queries above.

Event2: σ_i^* is a valid and nontrivial forgery.

Event3: Event2 occurs and $c_i^* = 1$, $d_i^* = 1$.

It is easy to see that $acc = \Pr[Event1 \wedge Event2 \wedge Event3] \geq \zeta^{q_{sv}} (1 - \zeta)^2 \varepsilon$. We can find the value of ζ that maximize this probability is $\zeta = \frac{q_{sv}}{2 + q_{sv}}$ and the maximized probability is $\frac{4\varepsilon}{e(q_{sv} + 2)^2}$. According to [44], we have

$$\begin{aligned} \xi &\geq acc \left(\frac{acc}{q_{H_1} + q_s + 1} - \frac{1}{q} \right) \\ &\geq \frac{4\varepsilon}{e(q_{sv} + 2)^2} \left[\frac{4\varepsilon}{e(q_{sv} + 2)^2 (q_{H_1} + q_s + 1)} - \frac{1}{q} \right] \end{aligned} \quad (4)$$

The running time t' required for C to solve the co-CDH problem can be approximated as twice the time it takes for C to simulate without aborting, plus the time required to compute the co-CDH solution. The time it takes for C to simulate without aborting can be calculate as $t + (q_{H_0} + q_{pk} + 5q_s)T_{m_1} + q_{H_2}T_{m_2} + O(q_{H_1} + q_s + 1)\bar{t}$. The time it takes for C to solve the co-CDH problem can be calculate as $3T_{m_1}$. Thus $t' \approx 2t + (2q_{H_0} + 2q_{pk} + 10q_s + 3)T_{m_1} + 2q_{H_2}T_{m_2} + O(q_{H_1} + q_s + 1)\bar{t}$.

Theorem 3: Our ES&PPS satisfies message authentication.

Proof: In our proposed scheme, before a vehicle V_i broadcasts a TRM M_i that contains a timestamp Ts_i , V_i first uses the algorithm $\text{Sign}(M_i, sk_{i,j}, PID_{i,j}, pk_{i,j})$ of our CLSS with batch verification to generate a signed TRM $\{PID_{i,j}, pk_{i,j}, M_i, \sigma_{i,j}\}$. When a vehicle V_j receives n signed TRMs $\{M_i, \sigma_{i,j}, PID_{i,j}, pk_{i,j}\}_{i=1}^n$, V_j could use the $\text{BatchVerify}(\{M_i, \sigma_{i,j}, PID_{i,j}, pk_{i,j}\}_{i=1}^n)$ algorithm to verify that validity of these signed TRMs. According to Theorems 1 and 2, the signed TRMs are unforgeable. Thus, our ES&PPS achieves message authentication.

Theorem 4: Our ES&SSP satisfies conditional privacy.

Proof: In our scheme, a vehicle V_i uses a pseudonym $PID_{i,j}$ to protect its true identity. This pseudonym is generated based on a secure symmetric key encryption scheme $E_{key}(\cdot)/D_{key}(\cdot)$, which takes as input V_i 's true identity UID_i and an index j . Without the knowledge of the symmetric key key , any entity cannot recover V_i 's true identity UID_i . Meanwhile, since the KGC has a knowledge of key , the KGC can recover the true identity UID_i of V_i . Thus, our ES&SSP achieves conditional privacy.

VII. CONCLUSION

We had described our proposed ES&SSP in VANETs based on a new provably secure CLSS with batch verification. Our security and performance evaluations also demonstrated that the scheme achieves the intended properties (i.e., message authentication, conditional privacy, certificate management freeness, fast response and key escrow freeness), as well as the scheme is more efficient in terms of computation and communication overheads compared to other existing provably secure solutions designed in CLPKC.

REFERENCES

- [1] L. Zhang, J. Li, and Y. Yang, "Message linkable group signature with information binding and efficient revocation for privacy-preserving announcement in VANETs," *IEEE Trans. Dependable Secure Comput.*, early access, doi: 10.1109/TDSC.2024.3381436.
- [2] R. Zhang, L. Zhang, Q. Wu, and J. Zhou, "Secure channel establishment scheme for task delivery in vehicular cloud computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2865–2880, Jan. 2024.
- [3] C. Wu, L. Zhang, L. Xu, K. R. Choo, and L. Zhong, "Privacy-preserving serverless federated learning scheme for Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 22429–22438, Jun. 2024.
- [4] A. Y. Dak, S. Yahya, and M. Kassim, "A literature survey on security challenges in VANETs," *Int. J. Comput. Theory Eng.*, vol. 4, no. 6, pp. 1007, 2012.
- [5] F. Qu, Z. Wu, F.-Y. Wang, and W. Cho, "A security and privacy review of VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 6, pp. 2985–2996, Dec. 2015.
- [6] L. Zhang, B. Kang, F. Dai, Y. Zhang, and H. Liu, "Hybrid and hierarchical aggregation-verification scheme for VANET," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 11189–11200, Oct. 2022.
- [7] W. Von, D. Alain, H. Soichi, D. Khadke, M. Rohan, and Q. Robert, "OSG PKI transition: Experiences and lessons learned," in *Proc. Int. Symp. Grids Clouds*, 2014, pp. 23–28.
- [8] L. Zhang, C. Hu, Q. Wu, J. Domingo-Ferrer, and B. Qin, "Privacy-preserving vehicular communication authentication with hierarchical aggregation and fast response," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2562–2574, Aug. 2016.
- [9] J. B. Kenney, "Dedicated short-range communications (DSRC) standards in the United States," in *Proc. IEEE*, vol. 99, no. 7, pp. 1162–1182, Jul. 2011.
- [10] A. Fiat, "Batch RSA," *J. Cryptol.*, vol. 10, no. 2, pp. 75–88, 1997.
- [11] D. Boneh, C. Gentry, B. Lynn, and H. Shacham, "Aggregate and verifiably encrypted signatures from bilinear maps," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Berlin, Germany, Springer, 2003, pp. 416–432.
- [12] J. Camenisch, S. Hohenberger, and M. Ø. Pedersen, "Batch verification of short signatures," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Berlin, Heidelberg, Germany, Springer, 2007, pp. 246–263.
- [13] N. Eltayieb, R. Elhabob, M. U. Aftab, R. Kuleev, M. Mazzara, and M. Ahmad, "Secure aggregate signature scheme for smart city applications," *Comput. Commun.*, vol. 193, pp. 388–395, 2022.
- [14] Y. Genc, N. Aytas, A. Akkoc, E. Afacan, and E. Yazgan, "ELCPAS: A new efficient lightweight certificateless conditional privacy preserving authentication scheme for IoV," *Veh. Commun.*, vol. 39, 2023, Art. no. 100549.
- [15] J. Zhang, J. Cui, H. Zhong, Z. Chen, and L. Liu, "PA-CRT: Chinese remainder theorem based conditional privacy-preserving authentication scheme in vehicular ad-hoc networks," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 2, pp. 722–735, Mar./Apr. 2021.
- [16] H. Xiong, J. Chen, Q. Mei, and Y. Zhao, "Conditional privacy-preserving authentication protocol with dynamic membership updating for VANETs," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 2089–2104, May/Jun. 2022.
- [17] J. P. Hubaux, S. Capkun, and J. Luo, "The security and privacy of smart vehicles," *IEEE Secur. Privacy*, vol. 2, no. 3, pp. 49–55, May/Jun. 2004.
- [18] D. He, S. Zeadally, B. Xu, and X. Huang, "An efficient identity-based conditional privacy-preserving authentication scheme for vehicular ad hoc networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2681–2691, Dec. 2015.
- [19] S.-F. Tzeng, S.-J. Horng, T. Li, X. Wang, P.-H. Huang, and M. K. Khan, "Enhancing security and privacy for identity-based batch verification scheme in VANETs," *IEEE Trans. Veh. Technol.*, vol. 66, no. 4, pp. 3235–3248, Apr. 2017.
- [20] C. Zhang, R. Lu, X. Lin, P.-H. Ho, and X. Shen, "An efficient identity-based batch verification scheme for vehicular sensor networks," in *Proc. 27th Conf. Comput. Commun.*, 2008, pp. 246–250.
- [21] Z. Gong, Y. Long, X. Hong, and K. Chen, "Two certificateless aggregate signatures from bilinear maps," in *Proc. 8th ACIS Int. Conf. Softw. Eng. Artif. Intell. Netw. Parallel/Distrib. Comput.*, 2007, pp. 188–193.
- [22] L. Zhang and F. Zhang, "A new certificateless aggregate signature scheme," *Comput. Commun.*, vol. 32, no. 6, pp. 1079–1085, 2009.
- [23] H. Xiong, Z. Guan, Z. Chen, and F. Li, "An efficient certificateless aggregate signature with constant pairing computations," *Inf. Sci.*, vol. 219, pp. 225–235, 2013.
- [24] L. Zhang, B. Qin, Q. Wu, and F. Zhang, "Efficient many-to-one authentication with certificateless aggregate signatures," *Comput. Netw.*, vol. 54, no. 14, pp. 2482–2491, 2010.
- [25] S. Batra and A. K. Malhi, "An efficient certificateless aggregate signature scheme for vehicular ad-hoc networks," *Discrete Math. Theor. Comput. Sci.*, vol. 17, no. 1, pp. 317–338, 2015.
- [26] P. Kumar and V. Sharma, "On the security of certificateless aggregate signature scheme in vehicular ad hoc networks," in *Proc. Int. Conf. Soft Comput. Theories Appl.*, Singapore, Springer, 2018, pp. 715–722.
- [27] S.-J. Horng, S.-F. Tzeng, P.-H. Huang, X. Wang, T. Li, and M. K. Khan, "An efficient certificateless aggregate signature with conditional privacy preserving for vehicular sensor networks," *Inf. Sci.*, vol. 317, pp. 48–66, 2015.
- [28] J. Cui, J. Zhang, H. Zhong, R. Shi, and Y. Xu, "An efficient certificateless aggregate signature without pairings for vehicular ad hoc networks," *Inf. Sci.*, vol. 451, pp. 1–15, Jul. 2018.
- [29] I. A. Kamil and S. O. Ogundoyin, "An improved certificateless aggregate signature scheme without bilinear pairings for vehicular ad hoc networks," *J. Inf. Secur. Appl.*, vol. 44, pp. 184–200, Feb. 2019.

- [30] Q. Mei, H. Xiong, J. Chen, M. Yang, S. Kumari, and M. K. Khan, "Efficient certificateless aggregate signature with conditional privacy preservation in IoV," *IEEE Syst. J.*, vol. 15, no. 1, pp. 245–256, Mar. 2021.
- [31] Y. Liang and Y. Liu, "Analysis and improvement of an efficient certificateless aggregate signature with conditional privacy preservation in VANETs," *IEEE Syst. J.*, vol. 17, no. 1, pp. 664–672, Mar. 2023.
- [32] Z. Gong, T. Gao, and N. Guo, "PCAS: Cryptanalysis and improvement of pairing-free certificateless aggregate signature scheme with conditional privacy-preserving for VANETs," *Ad Hoc Netw.*, vol. 144, 2023, Art. no. 103134.
- [33] Y. Han, W. Song, Z. Zhou, H. Wang, and B. Yuan, "eCLAS: An efficient pairing-free certificateless aggregate signature for secure VANET communication," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1637–1648, Mar. 2022.
- [34] H. Zheng, M. Luo, Y. Zhang, C. Peng, and Q. Feng, "A security-enhanced pairing-free certificateless aggregate signature for vehicular ad-hoc networks," *IEEE Syst. J.*, vol. 17, no. 3, pp. 3822–3833, Sep. 2023.
- [35] M. Geng and F. Zhang, "Batch verification for certificateless signature schemes," in *Proc. Int. Conf. Comput. Intell. Secur.*, 2009, pp. 288–292.
- [36] I. Ali, Y. Chen, N. Ullah, R. Kumar, and W. He, "An efficient and provably secure ECC-based conditional privacy-preserving authentication for vehicle-to-vehicle communication in VANETs," *IEEE Trans. Veh. Technol.*, vol. 70, no. 2, pp. 1278–1291, Feb. 2021.
- [37] X. Zhou, M. Luo, P. Vijayakumar, C. Peng, and D. He, "Efficient certificateless conditional privacy-preserving authentication for VANETs," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7863–7875, Jul. 2022.
- [38] M. Bellare, J. A. Garay, and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, Berlin, Germany, Springer, 1998, pp. 236–250.
- [39] S. D. Galbraith, K. G. Paterson, and N. P. Smart, "Pairings for cryptographers," *Discrete Appl. Math.*, vol. 156, no. 16, pp. 3113–3121, 2008.
- [40] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Berlin, Germany, Springer, 2003, pp. 452–473.
- [41] Accessed: Aug. 21, 2024. [Online]. Available: <https://www.keylength.com/en/4/>
- [42] L. Zhang, Q. Wu, A. Solanas, and J. Domingo-Ferrer, "A scalable robust authentication protocol for secure vehicular communications," *IEEE Trans. Veh. Technol.*, vol. 59, no. 4, pp. 1606–1617, May 2010.
- [43] Y. Yang, L. Zhang, Y. Zhao, K.-K. R. Choo, and Y. Zhang, "Privacy-preserving aggregation-authentication scheme for safety warning system in fog-cloud based VANET," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 317–331, 2022.
- [44] M. Bellare and G. Neven, "Multi-signatures in the plain public-key model and a general forking lemma," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2006, pp. 390–399.



Qian Wu received the BS degree from the School of Economics and Management, Anhui University of Science and Technology, China. She is currently working toward the master's degree with the School of Software Engineering, East China Normal University, China. Her research interests include VANET security and batch verification.



Lei Zhang (Member, IEEE) received the PhD degree in computer engineering from Universitat Rovira i Virgili, Tarragona, Spain. Since then, he has been with Universitat Rovira i Virgili, as a postdoctoral researcher. He is currently a full professor with the Software Engineering Institute, East China Normal University, Shanghai, China. He has been a holder/coholder of more than 20 China/Spain funded (key) projects. His fields of activity are information security, VANET security, cloud security, data privacy, and network security. He has authored more than 100 publications. He has served in the program committee of more than 100 international conferences in information security and privacy. He is also an editor of several international journals.



Yafang Yang received the PhD degree from the School of Computer Science and Technology, Fudan University, China, in 2022. She is currently with the School of Computer Science and Technology, Fudan University, China as a postdoctoral researcher. Her research interests include information security, (certificateless) public key cryptography, applied cryptography, and VANET security.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the PhD degree in information security from the Queensland University of Technology, Australia, in 2006. He currently holds the Cloud Technology Endowed Professorship with the University of Texas at San Antonio. He is the founding co-editor-in-chief of the *ACM Distributed Ledger Technologies: Research & Practice* and the founding chair of the IEEE TEMS Technical Committee (TC) on Blockchain and Distributed Ledger Technologies. He also serves as the department editor of the *IEEE Transactions on Engineering Management*, and the associate editor of the *IEEE Transactions on Dependable and Secure Computing* and *IEEE Transactions on Big Data*. He is the recipient of the 2022 IEEE Hyper-Intelligence TC Award for Excellence in Hyper-Intelligence Systems (Technical Achievement award), the 2022 IEEE TC on Homeland Security Research and Innovation Award, the 2022 IEEE TC on Secure and Dependable Measurement Mid-Career Award, and the 2019 IEEE TC on Scalable Computing Award for Excellence in Scalable Computing (Middle Career Researcher).